159102 workshop – Week 11 – Some selected solutions

1. Find Program 2 in the notes (to insert several nodes at the FRONT of a list). Modify the program to insert only three (3) nodes at the front of the list. Test the program.

Solution to Q1:

The class and the functions all remain unchanged. This is one of the main reasons for using functions, i.e. we can change a function (or main) and leave everything else unchanged. Change main to this:

```
int main() {      // insert at the FRONT of the list
  Node *temp;
  int number, i;
  listpointer = NULL;
  for (i = 0; i < 3; i++) {
    cout << "Enter an integer value ";
    cin >> number;
    temp = new Node;
    temp->loaddata(number, listpointer);
    listpointer = temp;
  }
  displaylist();
}
```

2. Find Program 3 in the notes (to insert several nodes at the BACK of a list). Modify the program to insert only three (3) nodes at the back of the list. Test the program.

Solution to Q2:

The class and the functions all remain unchanged. Change main to this:

```
int main() {      // insert at the BACK of the list
  Node *temp, *lastnode;
  int number, i;
  listpointer = NULL;
  // set up the first node
  cout << "Enter the first integer value ";
  cin >> number;
  listpointer = new Node;
  listpointer->loaddata(number, NULL);
  lastnode = listpointer;
  // insert the other nodes
  for (i = 1; i < 3; i++) {  // start from 1
    cout << "Enter an integer value ";
    cin >> number;
    temp = new Node;
    temp->loaddata(number, NULL);
    lastnode->setnext(temp);
    lastnode = temp;
  }
  displaylist();
}
```

3. Find Program 4 in the notes (to delete a node from a list). Modify the program to (a) insert four (4) nodes in the list, and then (b) remove the third node. Test the program.

Solution to Q3:

The class and the functions all remain unchanged. Change main to this:

```
int main() {
  Node *ptr1, *ptr2, * ptr3, *ptr4, *temp;
  int i;
  listpointer = NULL;
  // insert four nodes in the list
  for (i = 0; i < 4; i++) {
    temp = new Node;
    temp->loaddata(i * 10, listpointer);
    listpointer = temp;
  }
  displaylist();
  cout << endl;
  // now remove the third node
  ptr1 = listpointer;   // ptr1 points to the first node
  ptr2 = ptr1->getnext();  // the second node
  ptr3 = ptr2->getnext();  // the third node
  ptr4 = ptr3->getnext();  // the fourth node
  // change node2 to point to node4
  ptr2->setnext(ptr4);
  delete ptr3;  // delete node3 to free memory
  displaylist();
  cout << endl;
}
```

4.** Expand Program 4 in the notes to do the following:
a) enter five (5) values into the list
b) enter the value that must be deleted
c) find the value and delete it from the list
d) test the program (remember to test deleting the first or last value)

Solution to Q4:

The class and the functions all remain unchanged. Change main to this:

```
int main() {
  Node *temp, *nextnode, *prevnode, *current, *previous;
  int searchvalue, currentvalue, i;
  bool found;
  listpointer = NULL;
  // insert five nodes in the list – this could be a function
  for (i = 0; i < 5; i++) {
    temp = new Node;
    temp->loaddata(i * 10, listpointer);
    listpointer = temp;
```

```
  }
  displaylist();
  cout << endl;
  // enter the value that must be deleted
  cout << "Enter the value to be deleted ";
  cin >> searchvalue;
  // find the node with the search value
  found = false;
  previous = NULL;
  current = listpointer;
  while (current != NULL) {
    currentvalue = current->getvalue();
    if (currentvalue == searchvalue) {
      found = true;
      temp = current;
      prevnode = previous;
    }
    previous = current;
    current = current->getnext();
  }
  if (found == false) {
    cout << searchvalue << " is not in the list.\n";
    exit(2);
  }
  previous = prevnode;
  nextnode = temp->getnext();
  if (previous == NULL) {  // this is the first node
    listpointer = nextnode;
  } else {
    previous->setnext(nextnode);
  }
  delete temp;  // delete the node to free memory
  displaylist();
  cout << endl;
}
```

5. Create a class to hold data for a domestic flight. The class must store the following information:
the flight number, e.g. NZ161 (string)
the departure place, e.g. Auckland (string)
the arrival place, e.g. Wellington (string)
the travel time in minutes, e.g. 85 (positive whole number)
Create some useful methods, for example loaddata and display.
Using the `<list>` library, create a list where each node in the list will be an object of the flight class described above.

Solution to Q5:

Note – you are not required to provide a full program.

```
class flight_class {
private:
  string fltnum, departure, arrival;
  int minutes;
```

```
public:
  void loaddata();
  void display();
};

list<flight_class> mylist;


//-------------- methods for the flight_class ------------
void flight_class::loaddata() {
  cout << "Enter the flight number, e.g. NZ161 ";
  getline(cin, fltnum);
  cout << "Enter the departure place ";
  getline(cin, departure);
  cout << "Enter the arrival place ";
  getline(cin, arrival);
  cout << "Enter the travel time in minutes ";
  cin >> minutes;
  getchar();  // flush the input buffer
}

void flight_class::display() {
  cout << "Information about flight " << fltnum << endl;
  cout << "Departs from " << departure << endl;
  cout << "Arrives at " << arrival << endl;
  cout << "Travel time is " << minutes << " minutes.\n";
}
```

6. Use Program 5 in the notes (using the linked list library) and the information from Question 5 above. Write a program that reads in the data for three (3) flights and the displays the details of all 3 flights. The flight data must be stored in the list.

Solution to Q6:

```
#include <iostream>
#include <cstdio>  // for getchar
#include <list>
using namespace std;

class flight_class {
private:
  string fltnum, departure, arrival;
  int minutes;
public:
  void loaddata();
  void display();
};

void displaylist(list<flight_class> l);

list<flight_class> mylist;

int main() {
```

```
  int i;
  flight_class temp;
  for (i = 0; i < 3; i++) {
    temp.loaddata();
    mylist.push_back(temp);
  }
  displaylist(mylist);
}

void displaylist(list<flight_class> l) {
  list<flight_class>::iterator ptr;
  for (ptr = l.begin(); ptr != l.end(); ptr++) {
    ptr->display();
  }
  cout << endl;
}

//-------------- methods for the flight_class ------------
void flight_class::loaddata() {
  cout << "Enter the flight number, e.g. NZ161 ";
  getline(cin, fltnum);
  cout << "Enter the departure place ";
  getline(cin, departure);
  cout << "Enter the arrival place ";
  getline(cin, arrival);
  cout << "Enter the travel time in minutes ";
  cin >> minutes;
  getchar();  // flush the input buffer
}

void flight_class::display() {
  cout << "Information about flight " << fltnum << endl;
  cout << "Departs from " << departure << endl;
  cout << "Arrives at " << arrival << endl;
  cout << "Travel time is " << minutes << " minutes.\n";
}
```